

An introduction to AI

Machine Learning & Neural Networks

@ENSPIMA – Bordeaux INP

Jean-Luc.Charles@mailo.com

Sept. 2026



Welcome to the course

"An introduction to AI: Machine Learning & Neural Networks"



An introduction to get familiarised with...

- Machine learning (ML)
- Deep Learning (DL) with Artificial Neural Networks (ANN)
- Training & operating Artificial Neural Networks
- Applications to faults detection.

Welcome to the course

"An introduction to AI: Machine Learning & Neural Networks"



Who am I



Season 1

- I taught Python programming (Scientific & Object Oriented) for years at ENSAM
- I started to get interested in ML in 2015
- I have written materials about ML : workshop, project & practical work for engineering schools (ENSAM, ENSEIRB, ENSPIMA...)

Welcome to the course

"An introduction to AI: Machine Learning & Neural Networks"



Who am I



Season 1

- I taught Python programming (Scientific & Object Oriented) for years at ENSAM
- I started to get interested in ML in 2015
- I have written materials about ML : workshop, project & practical work for engineering schools (ENSAM, ENSEIRB, ENSPIMA...)



Season 2

- Since Sept. 2023 $\leadsto$ still working on *AI / Data processing* with Python as a freelancer.

Welcome to the course

"An introduction to AI: Machine Learning & Neural Networks"



Resources

- 2 lectures of 1h20
- 4 sessions of practical work "Deep Learning with Python" (4 x 3h) on **your laptop**
- Dedicated [GitHub repository](#) with all the course material (PDF, notebooks, videos...)

Welcome to the course

"An introduction to AI: Machine Learning & Neural Networks"



Resources

- 2 lectures of 1h20
- 4 sessions of practical work "Deep Learning with Python" (4 x 3h) on **your laptop**
- Dedicated [GitHub repository](#) with all the course material (PDF, notebooks, videos...)



Evaluation

- an individually assessed MCQ (coef. 1/4)
- a mini-project report completed in pairs (coef. 3/4).

Practical Work: 4 × 3h

Self training

- *Wake up your Python* with the 2 notebooks
- *AI & Deep Learning* with the 3 notebooks to target the skills:
 - load and pre-process MNIST images
 - build a **dense** neural network with [tensorflow](#) & [keras](#)
 - train the network to classify the MNIST images
 - evaluate/operate the trained network.

Practical Work: 4 × 3h

Self training

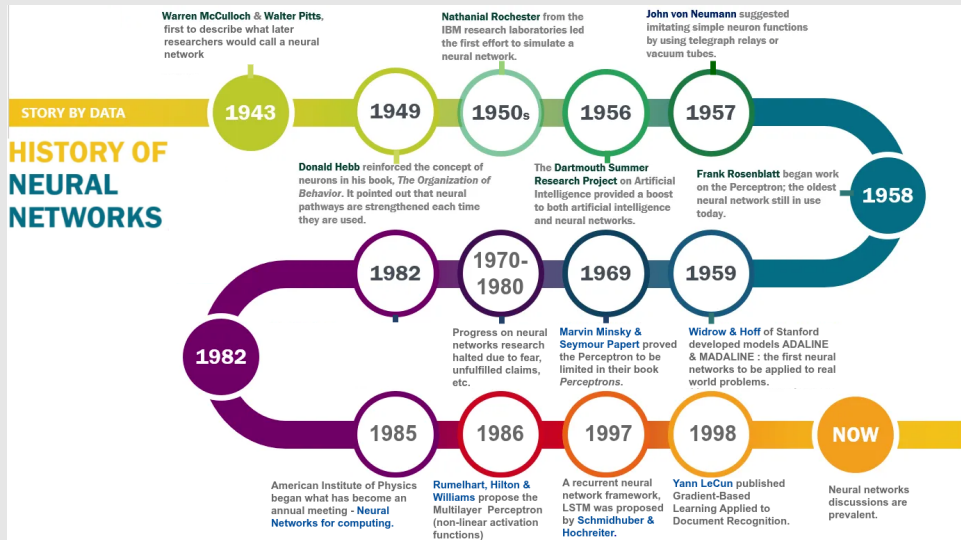
- *Wake up your Python* with the 2 notebooks
- *AI & Deep Learning* with the 3 notebooks to target the skills:
 - load and pre-process MNIST images
 - build a **dense** neural network with [tensorflow](#) & [keras](#)
 - train the network to classify the MNIST images
 - evaluate/operate the trained network.

Mini-project: application to Fault Detection

- Challenge your skills with a practical use case of fault detection on a motor test bench with a simple dense neural network...

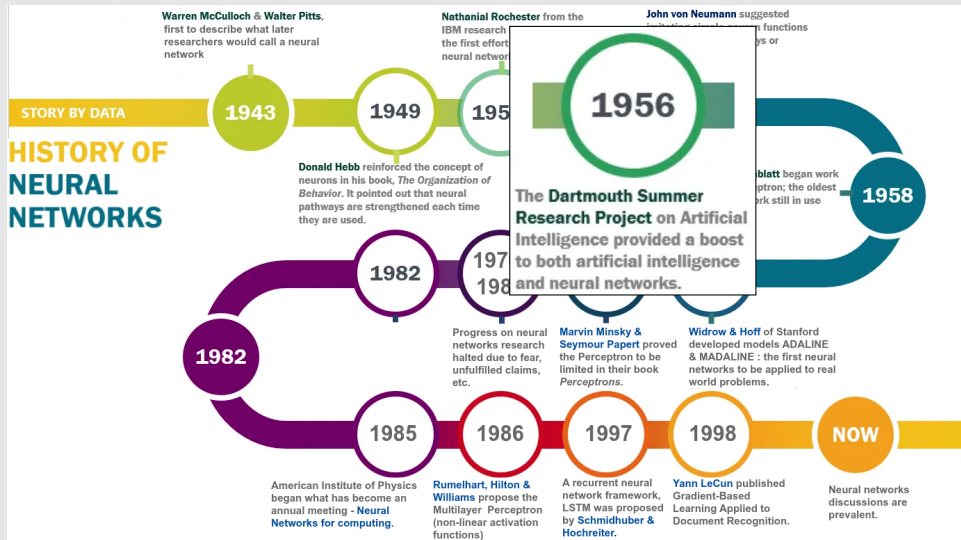
A brief history of AI and its main players

AI : brief history... from 1950 to 2000s



Adapted from : Kate Strachni: "Brief History of Neural Netowrks", medium.com

AI : brief history... from 1950 to 2000s



Adapted from : Kate Strachni: "Brief History of Neural Networks", medium.com

AI : brief history... from 1950 to 2000s

Warren McCulloch & Walter Pitts
first to describe what later
researchers would call a neural
network

Nathaniel Rochester from the

John von Neumann suggested

STORY BY DATA

1943

HISTORY OF NEURAL NETWORKS

Donald
neural
Behav
path
they

IN THIS BUILDING DURING THE SUMMER OF 1956,
JOHN MCCARTHY (DARTMOUTH COLLEGE), MARVIN L. MINSKY (MIT)
NATHANIEL ROCHESTER (IBM), AND CLAUDE SHANNON (BELL LABORATORIES)
CONDUCTED

THE DARTMOUTH SUMMER RESEARCH PROJECT ON ARTIFICIAL INTELLIGENCE

FIRST USE OF THE TERM "ARTIFICIAL INTELLIGENCE"

FOUNDING OF ARTIFICIAL INTELLIGENCE AS A RESEARCH DISCIPLINE

"To proceed on the basis of the conjecture
that every aspect of learning or any other feature of intelligence
can in principle be so precisely described that a machine can be made to simulate it."

IN COMMEMORATION OF THE PROJECT'S 50th ANNIVERSARY
JULY 13, 2006

1982

American Institute of Physics
began what has become an
annual meeting - **Neural
Networks for computing.**

Rumelhart, Hilton &
Williams propose the
Multilayer Perceptron
(non-linear activation
functions)

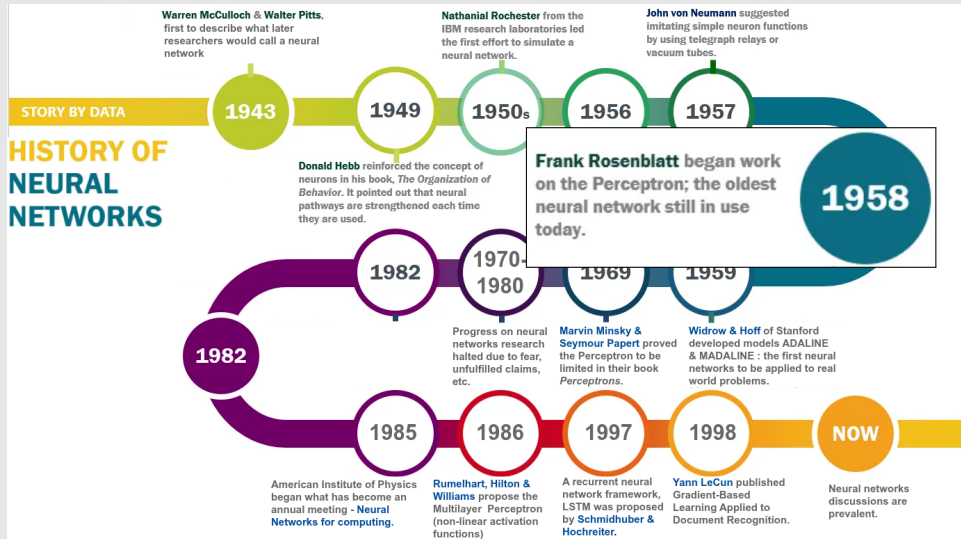
A recurrent neural
network framework,
LSTM was proposed
by **Schmidhuber &
Hochreiter.**

Yann LeCun published
Gradient-Based
Learning Applied to
Document Recognition.

Neural networks
discussions are
prevalent.

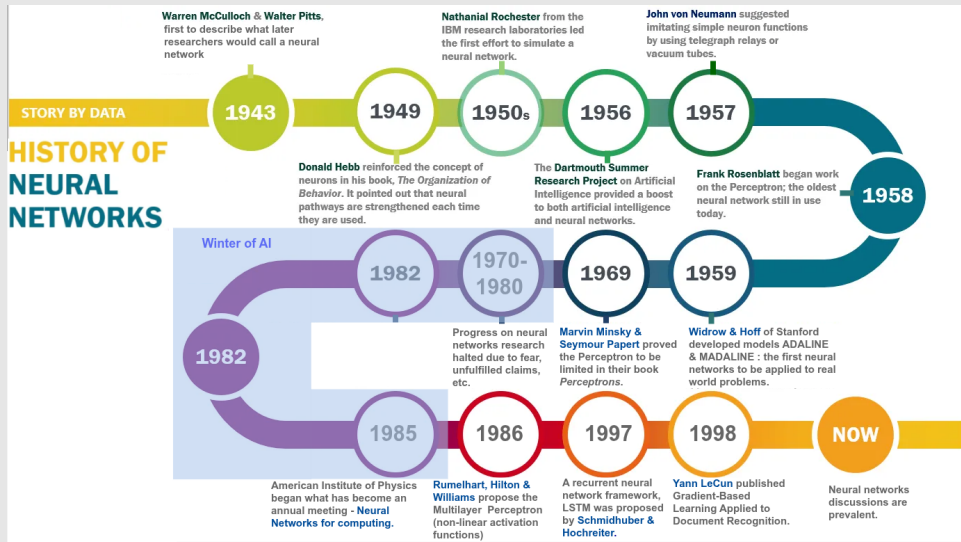
Adapted from : [Kate Strachni: "Brief History of Neural Netowrks", medium.com](#)

AI : brief history... from 1950 to 2000s



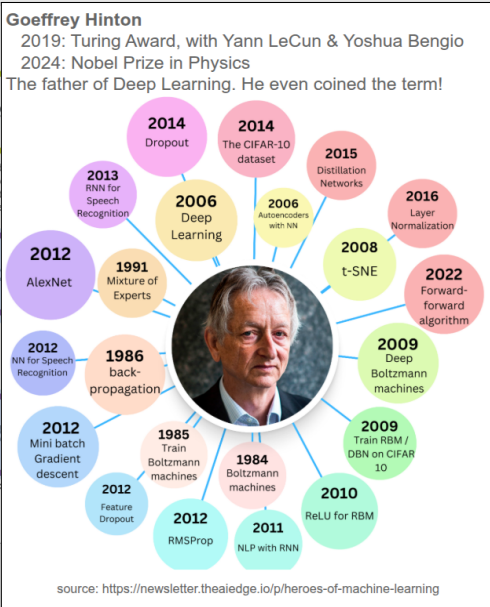
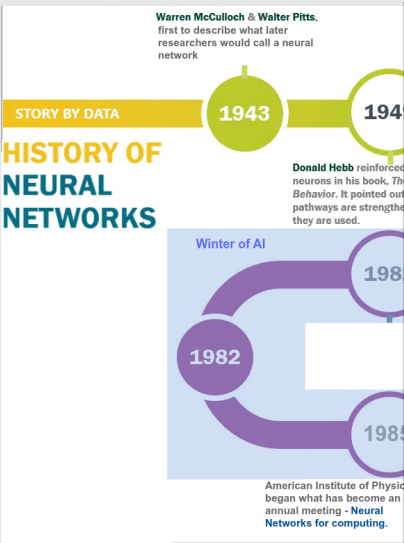
Adapted from : Kate Strachni: "Brief History of Neural Netowrks", medium.com

AI : brief history... from 1950 to 2000s



Adapted from : Kate Strachni: "Brief History of Neural Netowrks", medium.com

AI : brief history... from 1950 to 2000s



source: <https://newsletter.theaiedge.io/p/heroes-of-machine-learning>

AI : brief history... from 1950 to 2000s

STORY BY DATA

1943

Warren McCulloch & Walter Pitts,
first to describe what later
researchers would call a neural
network

HISTORY OF
NEURAL
NETWORKS

Donald Hebb
neurons in his
Behavior. It pe
pathways are
they are used.

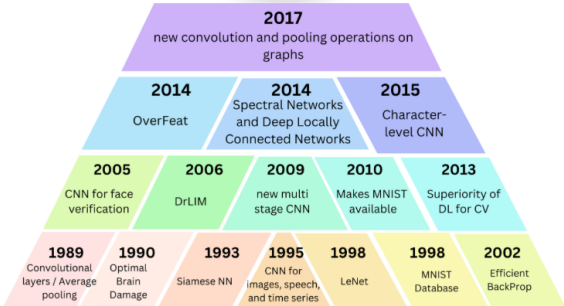
Winter of AI

1982

American Institute
began what has be
annual meeting - N
Networks for comp

Yann LeCun

2019: Turing Award, with Geoffrey Hinton & Yoshua Bengio



source: <https://newsletter.theaiedge.io/p/heroes-of-machine-learning>

AI : brief history... from 1950 to 2000s

Warren McCulloch & Walter Pitts, first to describe what later researchers would call a neural network

1943

1949 **Donald Hebb** reinforced the connections between neurons in his book, *The Organization of Behavior*. It pointed out that the neural pathways are strengthened as they are used.

1956 **Winter of AI**

1985 **American Institute of Physics** began what has become an annual meeting - **Neural Networks for computing**.

HISTORY OF NEURAL NETWORKS

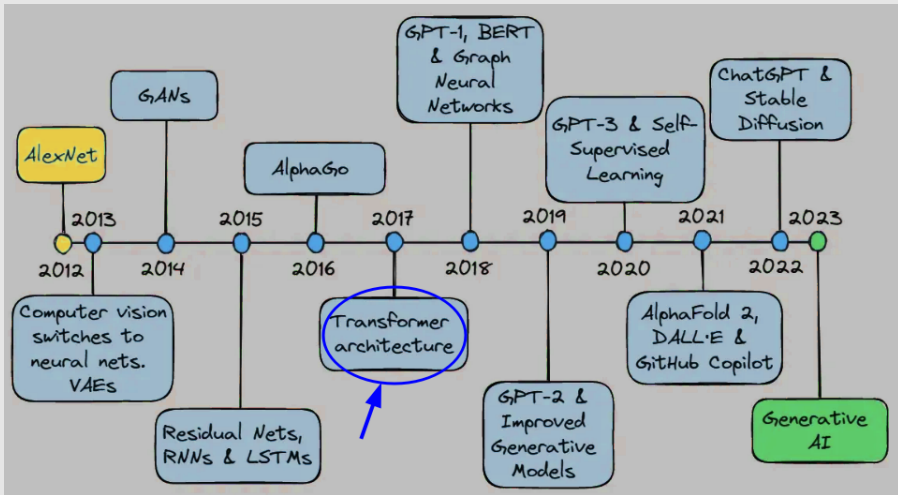
Yoshua Bengio

2019: Turing Award, with Geoffrey Hinton & Yann Lecun

- 2017:** CNN for Brain tumor
- 2015:** Graph attention networks
- 2015:** Visual Attention mechanism
- 2015:** BinaryConnect
- 2011:** ReLU
- 2011:** Bayesian hyper-parameter optimization
- 2010:** Xavier initialization
- 2010:** Stacked Denoising Autoencoders
- 2010:** curriculum learning
- 2009:** Vanishing gradient in RNN
- 1995:** CNN for images, speech, and time series
- 1998:** LeNet
- 2012:** Random search for hyper-parameters
- 2013:** Representation learning unification
- 2013:** Gradient clipping
- 2013:** Maxout networks
- 2006:** Greedy method for Deep Networks
- 2008:** Denoising Autoencoder
- 2014:** RNN Encoder-decoder
- 2014:** Gated recursive convolution
- 2014:** Bahdanau Attention
- 2014:** FitNets
- 2014:** Transfer learning

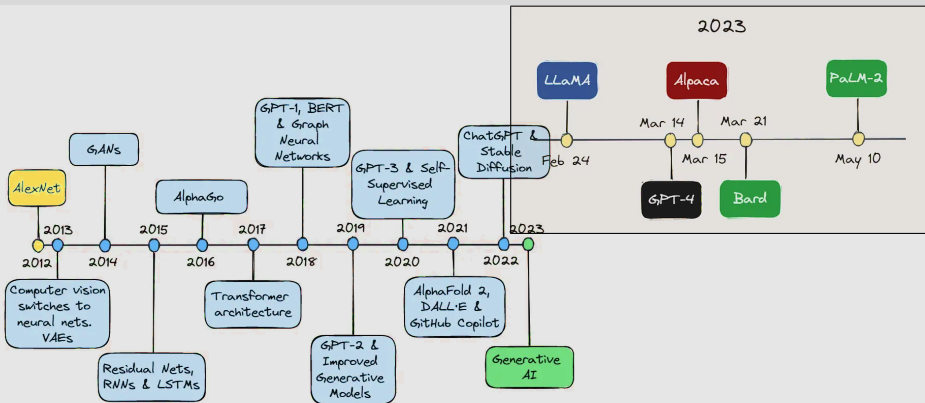
source: <https://newsletter.theaiedge.io/p/heroes-of-machine-learning>

AI : brief history... post 2012s: the “coming-of-age” of deep learning



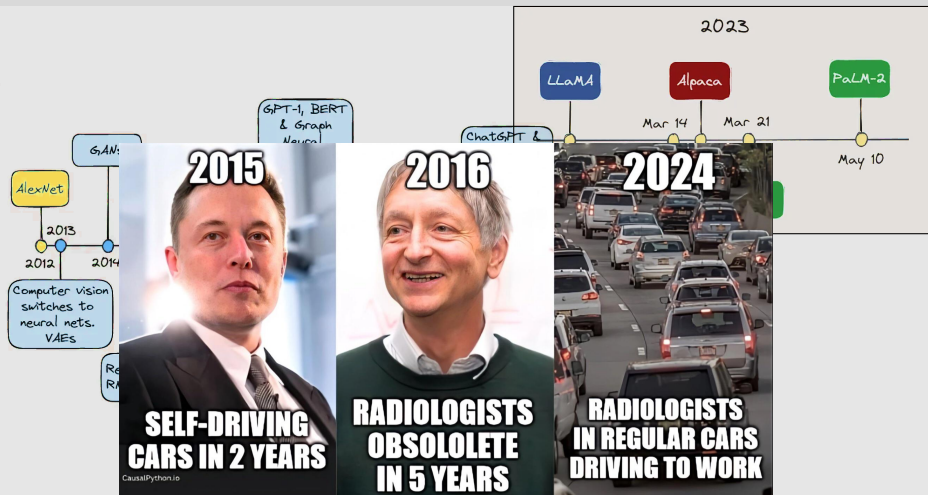
Source: [Thomas A Dorfe: "Ten Years of AI in Review", kdnuggets.com](https://kdnuggets.com/2023/01/ten-years-of-ai-in-review.html)

AI : brief history... post 2023s, crazy acceleration



Source: [Thomas A Dorfe: "Ten Years of AI in Review", kdnuggets.com](https://kdnuggets.com/2023/04/ten-years-of-ai-in-review.html)

AI : brief history... post 2023s, crazy acceleration



Artificial Intelligences ?

AI: Artificial Intelligences ?

Narrow/Weak AI

- Builds AI systems that **behave like humans** $\leadsto$ **AI Agent**.
- AI Agent are designed for **specific, narrow** tasks:
Email management, Productivity Tools, Generative AI...
(*AlphaGo, self-driving cars, robot systems in the medical field...*)
- The most common form of AI that we encounter today.
- **Agentic AI** [[Ran25](#)] [[Ahm25](#)] is "Weak AI with autonomy": it can plan and act toward a goal.

AI: Artificial Intelligences ?

Strong/General AI

- Builds systems with the ability to **reason in general**.
- Could understand, learn, and solve a very broad range of problems at approximately human level.
- **Whe are not yet here...** but closer and closer every day
- Do we really want that?

AI: Artificial Intelligences ?

Strong/General AI

- Builds systems with the ability to **reason in general**.
- Could understand, learn, and solve a very broad range of problems at approximately human level.
- **Whe are not yet here...** but closer and closer every day
- Do we really want that?
- On Sept. 8 , Jacob Coxon resigned from Anthropic:[Gla26]
he argues that leading AI companies are moving too quickly toward **self-improving superintelligence** without having solved fundamental safety and control problems. He said the AI industry was "gambling with our lives"...

AI: Societal issues

Explainability $\leadsto$ becoming a important aspect in research:

XAI **Explainable** Artificial Intelligence [MM25][Ali+23]

IML **Interpretable** Machine learning

[MCB20][Mol24][Rud+21]

AI: Societal issues

Explainability $\leadsto$ becoming an important aspect in research:

XAI **Explainable** Artificial Intelligence [MM25][Ali+23]

IML **Interpretable** Machine learning

[MCB20][Mol24][Rud+21]

Explainability

- **Unexplainability** of AI results still constitutes an obstacle to their dissemination today.
- Neural Networks (NN) are often denigrated as a **Black box** by scientists with a Cartesian approach.
- The growing complexity of NNs (LLM for example) makes the simple explanation of their outputs extremely difficult.

AI: Societal issues

Decision-making

- **Decisions** in sensible fields (justice, insurance, defense...) being ceded to ML algorithms to the detriment of human control
→ loss of fairness and equity.
- Decision-making algorithms rely on assumptions (quality of training data...) not always verified

AI: Societal issues

Decision-making

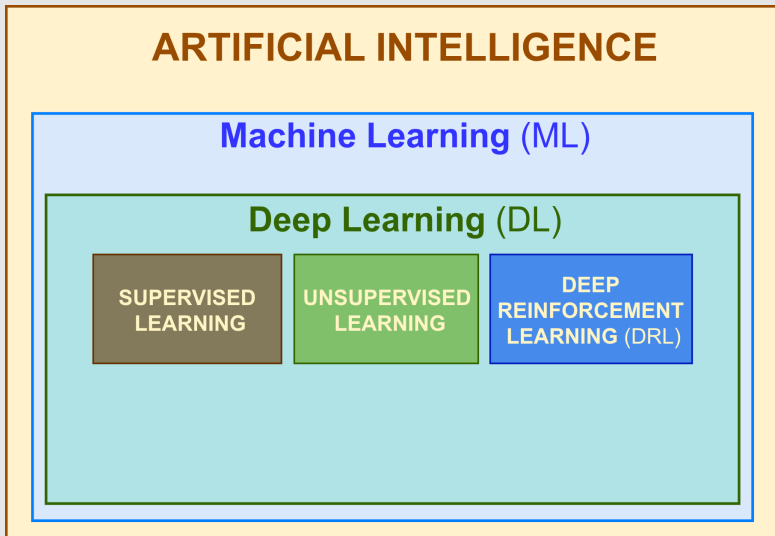
- **Decisions** in sensible fields (justice, insurance, defense...) being ceded to ML algorithms to the detriment of human control
→ loss of fairness and equity.
- Decision-making algorithms rely on assumptions (quality of training data...) not always verified

Certification

- The **evaluation** and **certification** of AI systems remains a major issue for their integration in the industry
- Formal certification of ML algorithms is still a subject of research
- Examples: [LNE: Process certification for AI](#),
[Fraunhofer: Auditing and Certification of AI Systems](#)

Machine Learning and Deep Learning: Fields of Artificial Intelligence

Machine Learning: a field of AI



Branches of Machine Learning

Supervised learning

A **labeled dataset** is used to train algorithms (models):

- **Classification**

- Images classification
- Objects detection in images
- Speech recognition...

- **Regression**

- Predict a value...

- **Anomaly detection**

- Spam detection
- Manufacturing: finding known (learned) defects
- Weather prediction
- Diseases classification...

...

Branches of Machine Learning

Unsupervised learning

The algorithm attempts to identify patterns / structures within **unlabeled datasets**:

- **Clustering & Grouping**

- Data mining, web data grouping, news grouping...
- Market segmentation
- Astronomical data analysis...

- **Anomaly Detection**

- Manufacturing: finding defects (even new ones)
- Monitoring abnormal activity: failure, hacker...
- Fraud detection: fake account on Internet...

- **Dimensionality reduction**

- Data compressions...

...

Branches of Machine Learning

Deep Reinforcement Learning DRL

An agent (the model) learns how to drive an environment within a trial and error process by maximising a cumulated **reward**:

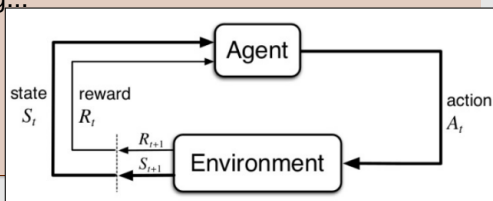
- **Control/command**

- Controlling drones, [mechatronic systems](#), [electric VTOL rocket](#), [robots](#)
- Factory optimization
- Financial (stock) trading...

- **Decision making**

- games (video games)
- financial analysis...

...



Various approaches for ML algorithms

Supervised learning:

- Neural Networks
- Bayesian inference
- Random forest
- Decision Tree
- Support Vector Machine (SVM)
- K-Nearest Neighbor (KNN)
- Linear regression
- Logistic regression...

Unsupervised learning:

- Neural Networks
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- K-mean & Prob. clustering...

Reinforcement learning:

- Neural Networks (Q-learning, Actor-Critic, DDPG, PPO...)
- Monte Carlo...

Self-Supervised learning:

- Neural Networks (SimCLR, Dino...)

Various approaches for ML algorithms

Supervised learning:

- Neural Networks
- Bayesian inference
- Random forest
- Decision Tree
- Support Vector Machine (SVM)
- K-Nearest Neighbor (KNN)
- Linear regression
- Logistic regression...

Unsupervised learning:

- Neural Networks
- Principal Component Analysis (PCA)
- Singular Value Decomposition (SVD)
- K-mean & Prob. clustering...

Reinforcement learning:

- Neural Networks
- Monte Carlo...

Self-Supervised learning:

- Neural Networks

A large variety of possibilities

→ success of **Deep Learning** with **Neural Networks**

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation[Tri26]
- ...



Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation[Tri26]
- ...

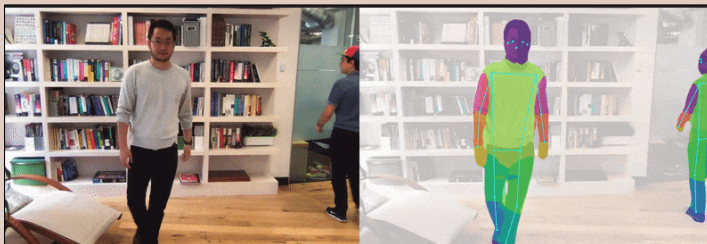


source : [Tensorflow tutorials object_detection](#)

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation

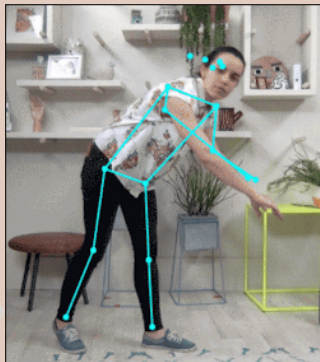


source : [Tensorflow](#)

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation[Tri26]
- ...

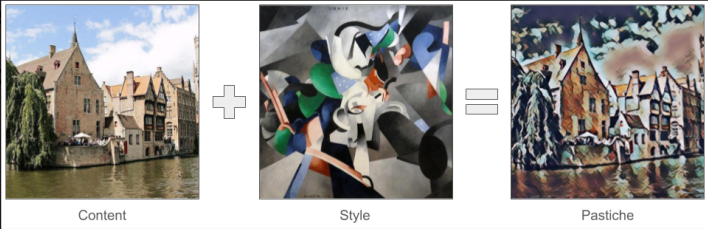


source : [Tensorflow pose_detection](#)

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation [Tri26]
- ...



source : [Tensorflow style_transfer](#)

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation[Tri26]
- ...



source : pyimagesearch.com

Fields & applications of DL

Computer Vision

- Image Classification
- Object Detection
- (Semantic) Segmentation
- Pose Estimation
- Style transfer
- OCR (Optical Character Recognition)
- Image Generation[[Tri26](#)]
- ...



source : openart.ai

Fields & applications of ML

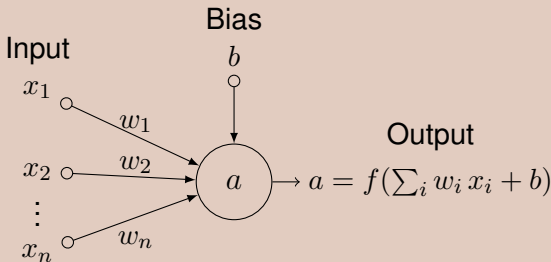
NLP (Natural Language Processing)

- Natural Language Understanding (NLU)
- Natural Language Generation (NLG)
- Speech recognition / Speech Synthesis (Text To Speech)
- Machine Translation (languages)
- Virtual agents and ChatBots
- LLM ChatBots
- ...

Neural Networks Technical aspects

NN technical aspects: ingredients

The artificial neurone



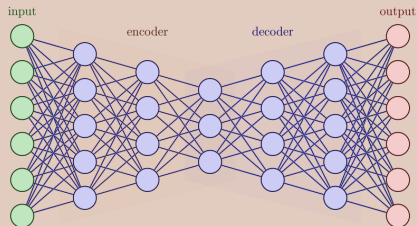
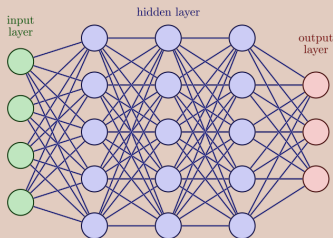
(Image by JLC)

- Receives the input stimuli $(x_i)_{i=1..n}$ affected by **weights** (w_i)
- Computes a **pre-activation** $z = \sum_i w_i x_i + b$
- Outputs its **activation** $a = f(z) = f(\sum_i w_i x_i + b)$, with f a non-linear **activation function**.

NN technical aspects: architectures

Neural network

Neurons are grouped by layers to form a **Neural Network**



source : tikz.net/neural_networks

NN technical aspects: architectures

Many architectures for many applications

- ***Dense Neural Network (DNN)***
The simplest architecture made of successive Fully Connected layers of neurones (aka FCNN: Fully Connected NN, or MLP: Multi Layer Perceptron)
- *Convolutional (CNN)*
- *Recurrent (RNN)*
- *Auto Encoder (AEN)*
- *Generative Adversarial (GAN)*
- *Transformers*
- *Large Language Model (LLM)*
- ...

NN technical aspects: architectures

Many architectures for many applications

- ***Dense Neural Network (DNN)***
- ***Convolutional (CNN)***
Neurons computing a convolution operation on data, mostly used for analyzing and classifying images.
- *Recurrent (RNN)*
- *Auto Encoder (AEN)*
- *Generative Adversarial (GAN)*
- *Transformers*
- *Large Language Model (LLM)*
- ...

NN technical aspects: architectures

Many architectures for many applications

- ***Dense Neural Network (DNN)***
- ***Convolutional (CNN)***
- ***Recurrent (RNN)***
Used for time series, like the Long Short-Term Memory (LSTM) algorithm.
- *Auto Encoder (AEN)*
- *Generative Adversarial (GAN)*
- *Transformers*
- *Large Language Model (LLM)*
- ...

NN technical aspects: architectures

Many architectures for many applications

- **Dense Neural Network (DNN)**
- **Convolutional (CNN)**
- **Recurrent (RNN)**
- **Auto Encoder (AEN)**
Dimensionality reduction, Feature extraction, Denoising of data/images, Inputting missing data...
- *Generative Adversarial (GAN)*
- *Transformers*
- *Large Language Model (LLM)*
- ...

NN technical aspects: architectures

Many architectures for many applications

- **Dense Neural Network (DNN)**
- **Convolutional (CNN)**
- **Recurrent (RNN)**
- **Auto Encoder (AEN)**
- **Generative Adversarial (GAN)**
to generate text, images, music...
- *Transformers*
- *Large Language Model (LLM)*
- ...

NN technical aspects: architectures

Many architectures for many applications

- ***Dense Neural Network (DNN)***
- ***Convolutional (CNN)***
- ***Recurrent (RNN)***
- ***Auto Encoder (AEN)***
- ***Generative Adversarial (GAN)***
- ***Transformers***
(2017) Natural language processing, image classification...
- ***Large Language Model (LLM)***
- ...

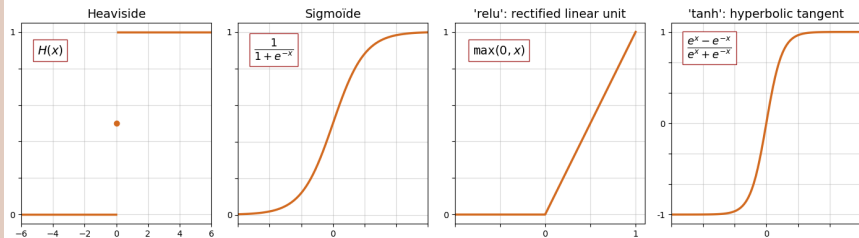
NN technical aspects: architectures

Many architectures for many applications

- **Dense Neural Network** (DNN)
- **Convolutional** (CNN)
- **Recurrent** (RNN)
- **Auto Encoder** (AEN)
- **Generative Adversarial** (GAN)
- **Transformers**
- **Large Language Model** (LLM)
read text, sound, write books, images, speak, make music ...
*Alibaba: Qwen - Anthropic: Claude - DeepSeek AI: DeepSeek -
Google DeepMind: Gemini - Open AI: ChatGPT - Perplexity AI:
Perplexity - Mistral: mistral - Meta: LLaMa -xAI: Grok...*
- ...

NN technical aspects: ingredients

Common activation functions

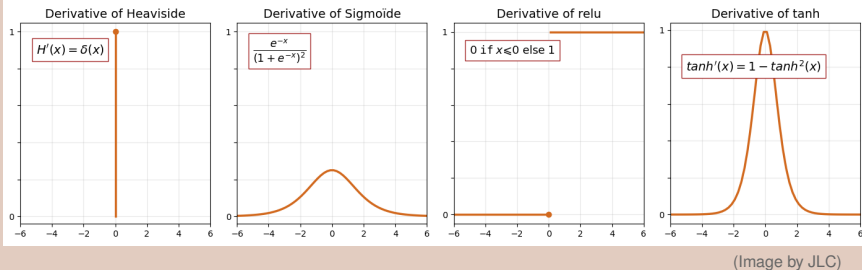


(Image by JLC)

- Introduces a **non-linear** behavior.
- Sets the **range** of the neuron activation: $[-1, 1]$, $[0, 1]$, $[0, \infty[...$
- The **bias** b sets the activation threshold of the neuron.
- Derivative of activation functions play an important role in **gradient descent** calculations ($\leadsto$ *backprop* algorithm).

NN technical aspects: ingredients

Common activation functions



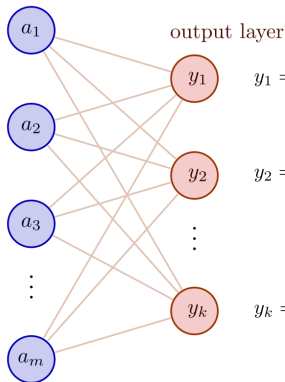
- Introduces a **non-linear** behavior.
- Sets the **range** of the neuron activation: $[-1, 1]$, $[0, 1]$, $[0, \infty[...$
- The **bias** b sets the activation threshold of the neuron.
- Derivative of activation functions play an important role in **gradient descent** calculations ($\leadsto$ *backprop* algorithm).

NN technical aspects: ingredients

Softmax

Activation function for **classifying**

hidden layer



$$z_i = \sum_{j=1}^m \omega_{ij} a_j$$

$$y_1 = \text{softmax}(z_1) = \frac{e^{z_1}}{\sum_{i=1}^k e^{z_i}}$$

$$y_2 = \text{softmax}(z_2) = \frac{e^{z_2}}{\sum_{i=1}^k e^{z_i}}$$

$$y_k = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{i=1}^k e^{z_i}}$$

(Image: JLC)

softmax activation y_i is like a probability:

- $y_i \in [0, 1]$
- $\sum_i y_i = 1$

↪ The neuron of highest probability gives the NN response.

NN technical aspects: ingredients

Activation functions

- Intermediate (hidden) layers $\leadsto$ **relu** promotes learning
- Output layer for **Classifying** $\leadsto$ **softmax** always used
- Python modules offer numerous activation functions:
 - tensorflow activation functions
 - pytorch activation functions

NN technical aspects: ingredients

OneHot coding

Label encoding (multi-class Classification)

- A classifying output layer $\leadsto$ a **vector** of softmax *probabilities*
 $\mathbf{y} = (y_i)_{i=1..k}$ with $y_i \in [0, 1]$
- Supervised Learning: class label $\leadsto$ scalar $l \in \mathbb{N}$
 $l \equiv$ the rank of the class in the class list
- How to compare a **vector** $\mathbf{y}$ with a **scalar** l ?

NN technical aspects: ingredients

OneHot coding

Label encoding (multi-class Classification)

- A classifying output layer $\leadsto$ a **vector** of softmax *probabilities* $\mathbf{y} = (y_i)_{i=1..k}$ with $y_i \in [0, 1]$
- Supervised Learning: class label $\leadsto$ scalar $l \in \mathbb{N}$
 $l \equiv$ the rank of the class in the class list
- How to compare a **vector** $\mathbf{y}$ with a **scalar** l ?
- Solution: **one-hot** encode the scalar l into a vector $\mathbf{r} \in \mathbb{R}^k$

$$\boxed{\mathbf{r} = (\delta_{i,l})_{i=1..k}} \quad \delta_{i,l} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{if } i \neq l \end{cases}$$

NN technical aspects: ingredients

OneHot coding

Label encoding (multi-class Classification)

- A classifying output layer $\leadsto$ a **vector** of softmax *probabilities* $\mathbf{y} = (y_i)_{i=1..k}$ with $y_i \in [0, 1]$
- Supervised Learning: class label $\leadsto$ scalar $l \in \mathbb{N}$
 $l \equiv$ the rank of the class in the class list
- How to compare a **vector** $\mathbf{y}$ with a **scalar** l ?

- Solution: **one-hot** encode the scalar l into

$$\mathbf{r} = (\delta_{i,l})_{i=1..k} \quad \delta_{i,l} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{if } i \neq l \end{cases}$$

class	one-hot(class)
apple	[1 0 0 0 0]
banana	[0 1 0 0 0]
orange	[0 0 1 0 0]
flower	[0 0 0 1 0]
car	[0 0 0 0 1]
bcycle	[0 0 0 0 0 1]

NN technical aspects: ingredients

OneHot coding

Label encoding (multi-class Classification)

- A classifying output layer $\leadsto$ a **vector** of softmax *probabilities* $\mathbf{y} = (y_i)_{i=1..k}$ with $y_i \in [0, 1]$
- Supervised Learning: class label $\leadsto$ scalar $l \in \mathbb{N}$
 $l \equiv$ the rank of the class in the class list
- How to compare a **vector** $\mathbf{y}$ with a **scalar** l
- Solution: **one-hot** encode the scalar l into

$$\mathbf{r} = (\delta_{i,l})_{i=1..k} \quad \delta_{i,l} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{if } i \neq l \end{cases}$$

r	One-hot(r)
0	[1 0 0 0 0 0 0 0 0 0]
1	[0 1 0 0 0 0 0 0 0 0]
2	[0 0 1 0 0 0 0 0 0 0]
3	[0 0 0 1 0 0 0 0 0 0]
4	[0 0 0 0 1 0 0 0 0 0]
5	[0 0 0 0 0 1 0 0 0 0]
6	[0 0 0 0 0 0 1 0 0 0]
7	[0 0 0 0 0 0 0 1 0 0]
8	[0 0 0 0 0 0 0 0 1 0]
9	[0 0 0 0 0 0 0 0 0 1]

NN technical aspects: ingredients

OneHot coding

Label encoding (multi-class Classification)

- A classifying output layer $\leadsto$ a **vector** of softmax *probabilities* $\mathbf{y} = (y_i)_{i=1..k}$ with $y_i \in [0, 1]$
- Supervised Learning: class label $\leadsto$ scalar $l \in \mathbb{N}$
 $l \equiv$ the rank of the class in the class list
- How to compare a **vector** $\mathbf{y}$ with a **scalar** l ?
- Solution: **one-hot** encode the scalar l into a vector $\mathbf{r} \in \mathbb{R}^k$

$$\mathbf{r} = (\delta_{i,l})_{i=1..k} \quad \delta_{i,l} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{if } i \neq l \end{cases}$$
- Class label l can be retrieved as $l = \operatorname{argmax}(\mathbf{r})$

NN technical aspects: ingredients

OneHot coding

- A classifying

$$y = (y_i)_{i=1..l}$$

- Supervised

$l \equiv$ the rank

- How to compute

- Solution: one

$$r = (\delta_{i,l})_{i=1..l}$$

- Class label l

tensorflow implementation of OneHot coding

```
import numpy as np
import tensorflow
from tensorflow.keras.utils import to_categorical
```

```
to_categorical(3)
```

```
array([0., 0., 0., 1.])
```

```
M = to_categorical([3,4,5])
M
```

```
array([[0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 1.]])
```

numpy argmax function

```
# rank of the max along columns -> axis=1
np.argmax(M, axis=1)
```

```
array([3, 4, 5])
```

NN technical aspects: ingredients

Cross-entropy error

Loss function for classifying

Supervised Learning classifying k classes ...

- $\mathbf{y} \leadsto$ vector of the true labels of a sample data (onehot coded)
- $\hat{\mathbf{y}} \leadsto$ vector predicted by the NN for the sample data
- How to compute the **error** between $\mathbf{y}$ and $\hat{\mathbf{y}}$?

NN technical aspects: ingredients

Cross-entropy error

Loss function for classifying

Supervised Learning classifying k classes ...

- $\mathbf{y} \leadsto$ vector of the true labels of a sample data (onehot coded)
- $\hat{\mathbf{y}} \leadsto$ vector predicted by the NN for the sample data
- How to compute the **error** between $\mathbf{y}$ and $\hat{\mathbf{y}}$?

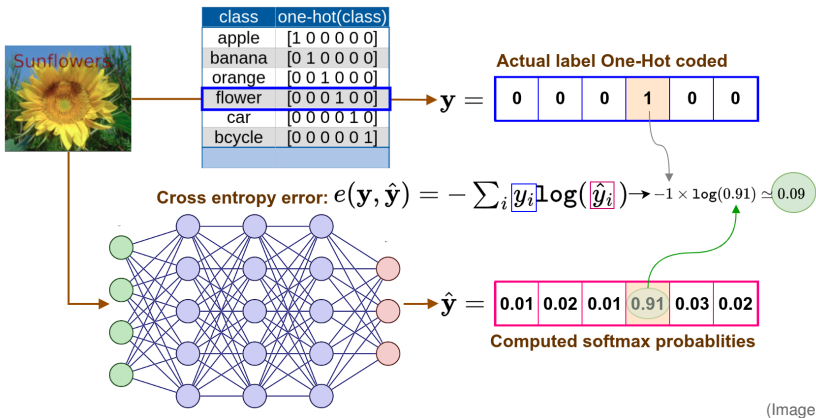
- $\leadsto$ **Cross-entropy error**

$$e(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i^k y_i \log(\hat{y}_i)$$

NN technical aspects: ingredients

Cross-entropy error

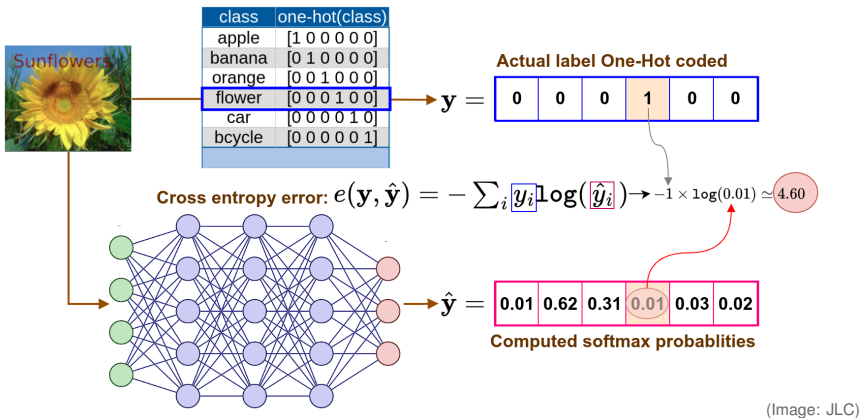
Loss function for classifying



NN technical aspects: ingredients

Cross-entropy error

Loss function for classifying



NN technical aspects: algorithms

Feed Forward

Maths formulation

~ Propagates the inputs in a forward direction, layer-by-layer, to generate the output.

NN technical aspects: algorithms

Feed Forward

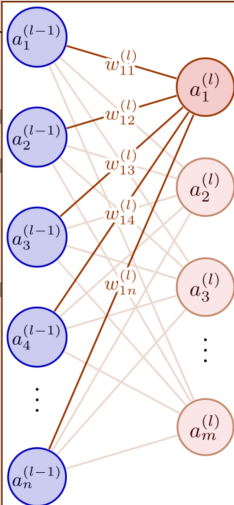
Maths formulation

- $\leadsto$ Propagates the inputs in a forward direction, layer-by-layer, to generate the output.
- layer $(0) \leadsto$ input layer, layer $(L) \leadsto$ output layer
- $\square_i^{(l)}$ refers to neuron i in layer l :
 - $b_i^{(l)} \leadsto$ bias term of neuron i in layer l
 - $f^{(l)} \leadsto$ activation function in layer l
- $w_{ij}^{(l)}$ the weight between neuron i in layer (l) and neuron j in layer $(l - 1)$

NN technical aspects: algorithms

Feed Forward

Maths formulation



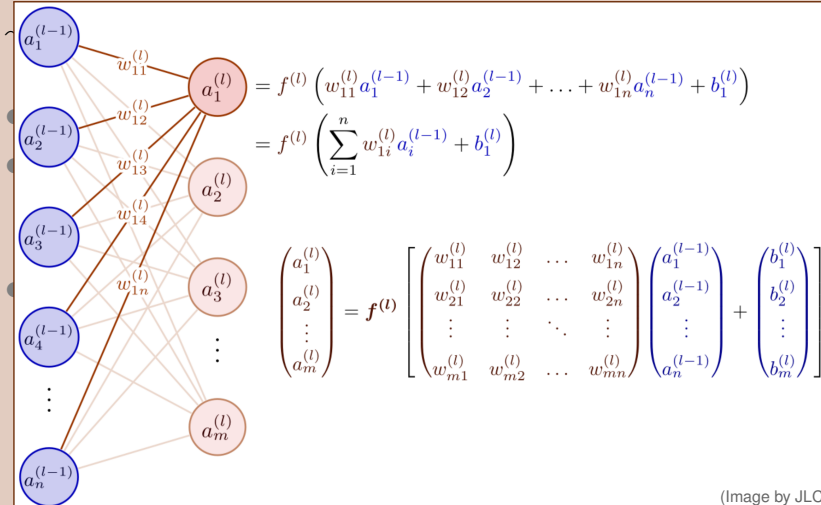
$$\begin{aligned}
 &= f^{(l)} \left(w_{11}^{(l)} a_1^{(l-1)} + w_{12}^{(l)} a_2^{(l-1)} + \dots + w_{1n}^{(l)} a_n^{(l-1)} + b_1^{(l)} \right) \\
 &= f^{(l)} \left(\sum_{i=1}^n w_{1i}^{(l)} a_i^{(l-1)} + b_1^{(l)} \right)
 \end{aligned}$$

(Image by JLC)

NN technical aspects: algorithms

Feed Forward

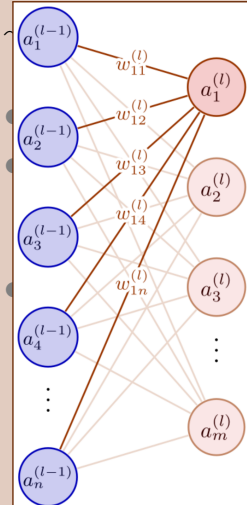
Maths formulation



NN technical aspects: algorithms

Feed Forward

Maths formulation



$$a_1^{(l)} = f^{(l)} \left(w_{11}^{(l)} a_1^{(l-1)} + w_{12}^{(l)} a_2^{(l-1)} + \dots + w_{1n}^{(l)} a_n^{(l-1)} + b_1^{(l)} \right)$$

$$= f^{(l)} \left(\sum_{i=1}^n w_{1i}^{(l)} a_i^{(l-1)} + b_1^{(l)} \right)$$

$$\begin{pmatrix} a_1^{(l)} \\ a_2^{(l)} \\ \vdots \\ a_m^{(l)} \end{pmatrix} = f^{(l)} \left[\begin{pmatrix} w_{11}^{(l)} & w_{12}^{(l)} & \dots & w_{1n}^{(l)} \\ w_{21}^{(l)} & w_{22}^{(l)} & \dots & w_{2n}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1}^{(l)} & w_{m2}^{(l)} & \dots & w_{mn}^{(l)} \end{pmatrix} \begin{pmatrix} a_1^{(l-1)} \\ a_2^{(l-1)} \\ \vdots \\ a_n^{(l-1)} \end{pmatrix} + \begin{pmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_m^{(l)} \end{pmatrix} \right]$$

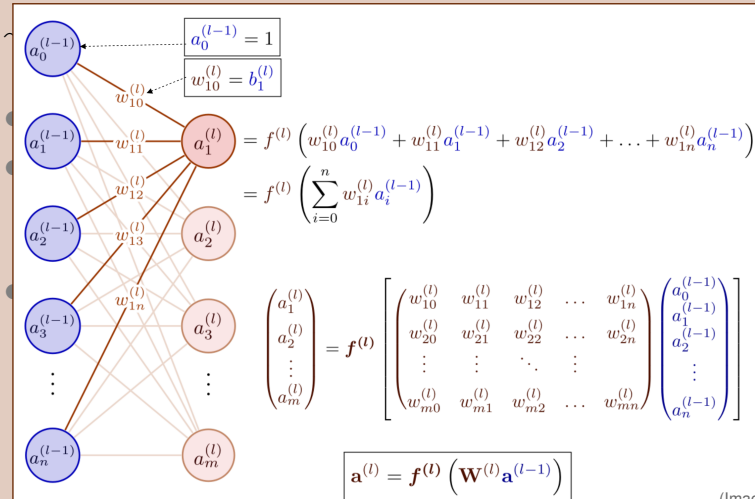
$$\mathbf{a}^{(l)} = f^{(l)} \left(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right)$$

(Image by JLC)

NN technical aspects: algorithms

Feed Forward

Maths formulation



(Image by JLC)

NN technical aspects: algorithms

Feed Forward

Maths formulation

- $\leadsto$ Propagates the inputs in a forward direction, layer-by-layer, to generate the output.
- layer $(0) \leadsto$ input layer, layer $(L) \leadsto$ output layer
- $i^{(l)}$ refers to neuron i in layer l .

$$\mathbf{y} = f^{(L)}(W^{(L)} \underbrace{f^{(L-1)}(\underbrace{W^{(L-1)} \dots f^{(2)}(\underbrace{W^{(2)} \underbrace{f^{(1)}(\underbrace{W^{(1)} \underbrace{x}_{a^{(0)}}}_{a^{(1)}}}_{a^{(2)}}}_{a^{(L-1)}}))}_{z^{(1)}}))}_{z^{(2)}}))$$

NN technical aspects: algorithms

Feed Forward

Python implementation

- Many Python modules (**tensorflow**, **pytorch**, **scikit-learn** ...) offer an efficient implementation of FeedForward
- Calculations (matrix-matrix & matrix-vector product...)
 - ~ specialized Python module **numpy**, **scipy** ...
 - ~ sub-modules of **tensorflow**, **pytorch** ...
- At the lowest level, optimized **Linear Algebra** libraries make the calculus ~ **BLAS**, **Lapack**
- Manufacturers **Intel**, **AMD** have their own hardware-optimized Linear Algebra library: **MKL** for intel, **AOCL-BLIS** for AMD.

NN technical aspects: algorithms

Feed Forward

Python implementation

- Many Python modules ([tensorflow](#), [pytorch](#), [scikit-learn](#) ...) offer an efficient implementation of FeedForward

tensorflow implementation of a sequential dense neural network

```
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

model = Sequential()
model.add(Input(shape=(100,))) # INPUT layer
model.add(Dense(64, activation='relu')) # HIDDEN layer
# ...
model.add(Dense(10, activation='softmax')) # OUTPUT layer
```

- Manufacturers [Intel](#), [AMD](#) have their own hardware-optimized Linear Algebra library: [MKL](#) for intel, [AOCL-BLIS](#) for AMD.

NN technical aspects: algorithms

Feed Forward

Python implementation

- Many Python modules ([tensorflow](#), [pytorch](#), [scikit-learn](#) ...) offer an efficient implementation of FeedForward
- Calculations (matrix-matrix & matrix-vector product...)
 - ↪ specialized Python module [numpy](#), [scipy](#) ...
 - ↪ sub-modules of [tensorflow](#), [pytorch](#) ...
- At the lowest level, optimized **Linear Algebra** libraries make the calculus ~ [BLAS](#), [Lapack](#)
- Manufacturers **Intel**, **AMD** have their own hardware-optimized Linear Algebra library: [MKL](#) for intel, [AOCL-BLIS](#) for AMD.

NN technical aspects: algorithms

Feed Forward

Python implementation

- Many Python modules ([tensorflow](#), [pytorch](#), [scikit-learn](#) ...) offer an efficient implementation of FeedForward
- Calculations (matrix-matrix & matrix-vector product...)
 - ↪ specialized Python module [numpy](#), [scipy](#) ...
 - ↪ sub-modules of [tensorflow](#), [pytorch](#) ...
- At the lowest level, optimized **Linear Algebra** libraries make the calculus ↪ [BLAS](#), [Lapack](#)
- Manufacturers [Intel](#), [AMD](#) have their own hardware-optimized Linear Algebra library: [MKL](#) for intel, [AOCL-BLIS](#) for AMD.

NN technical aspects: algorithms

Feed Forward

Python implementation

- Many Python modules ([tensorflow](#), [pytorch](#), [scikit-learn](#) ...) offer an efficient implementation of FeedForward
- Calculations (matrix-matrix & matrix-vector product...)
 - ↪ specialized Python module [numpy](#), [scipy](#) ...
 - ↪ sub-modules of [tensorflow](#), [pytorch](#) ...
- At the lowest level, optimized **Linear Algebra** libraries make the calculus ↪ [BLAS](#), [Lapack](#)
- Manufacturers **Intel**, **AMD** have their own hardware-optimized Linear Algebra library: [MKL](#) for intel, [AOCL-BLIS](#) for AMD.

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- **Goal** minimize a **cost function** $C(\theta)$ which depends on θ
 θ : collection of all the NN parameters (weights, biases...)
- **Strategy** Follow the **Gradient Descent** algorithm to find out a minimum of the cost function

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- **Goal** minimize a **cost function** $C(\theta)$ which depends on θ
 θ : collection of all the NN parameters (weights, biases...)

Supervised Learning: C is the **loss function** $L(\hat{y}, y)$ measuring the difference between true label y and predicted label $\hat{y}$

- **Strategy** Follow the **Gradient Descent** algorithm to find out a minimum of the cost function

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- **Goal** minimize a **cost function** $C(\theta)$ which depends on θ
 θ : collection of all the NN parameters (weights, biases...)

Supervised Learning: C is the **loss function** $L(\hat{y}, y)$ measuring the difference between true label y and predicted label $\hat{y}$

- **Strategy** Follow the **Gradient Descent** algorithm to find out a minimum of the cost function

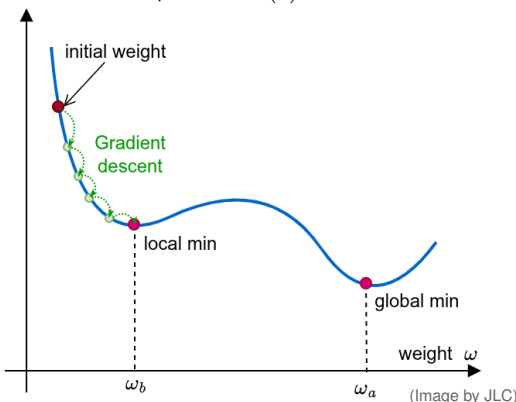
NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- Goal** minimize a **cost function** $C(\theta)$ which depends on θ (weights, biases...)

Cost function of 1 parameter: $C(\omega)$



measuring the

to find out

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- **Goal** minimize a **cost function** $C(\theta)$ which depends on θ
 θ : collection of all the NN parameters (weights, biases...)

Supervised Learning: C is the **loss function** $L(\hat{y}, y)$ measuring the difference between true label y and predicted label $\hat{y}$

- **Strategy** Follow the **Gradient Descent** algorithm to find out a minimum of the cost function

$$\forall \theta \in \theta: \boxed{\theta \leftarrow \theta - \alpha \frac{\partial C}{\partial \theta}} \quad \alpha \rightsquigarrow \text{the learning rate}$$

- **Classification** $\rightsquigarrow$ compute the gradient of the (cross-entropy) error $\nabla e(y, \hat{y})$ w.r.t. all weights $w_{ij}^{(l)}$ and biases $b_i^{(l)}$ of the NN

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- **Goal** minimize a **cost function** $C(\theta)$ which depends on θ
 θ : collection of all the NN parameters (weights, biases...)

Supervised Learning: C is the **loss function** $L(\hat{y}, y)$ measuring the difference between true label y and predicted label $\hat{y}$

- **Strategy** Follow the **Gradient Descent** algorithm to find out a minimum of the cost function

$$\forall \theta \in \theta: \boxed{\theta \leftarrow \theta - \alpha \frac{\partial C}{\partial \theta}} \quad \alpha \rightsquigarrow \text{the learning rate}$$

- **Classification** $\rightsquigarrow$ compute the gradient of the (cross-entropy) error $\nabla e(\mathbf{y}, \hat{\mathbf{y}})$ w.r.t. all weights $w_{ij}^{(l)}$ and biases $b_i^{(l)}$ of the NN

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- Delta rule [Yeh23]

$$\frac{\partial e(\mathbf{y}, \hat{\mathbf{y}})}{\partial \omega_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

$$\delta_i^{(l)} = a_j^{(l-1)} f'^{(l)}(z_i^l) \sum_{j \in N_i^{(l+1)}} \delta_j^{(l+1)} \omega_{ji}^{(l+1)}$$

$$N_i^{(l+1)} = \{ j \text{ indices in layer } (l+1) \mid z_j^{(l+1)} \text{ is connected to } a_i^{(l)} \}$$

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- Delta rule [Yeh23]

$$\frac{\partial e(\mathbf{y}, \hat{\mathbf{y}})}{\partial \omega_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

$$\delta_i^{(l)} = a_j^{(l-1)} f'^{(l)}(z_i^l) \sum_{j \in N_i^{(l+1)}} \delta_j^{(l+1)} \omega_{ji}^{(l+1)}$$

$$N_i^{(l+1)} = \{ j \text{ indices in layer } (l+1) \mid z_j^{(l+1)} \text{ is connected to } a_i^{(l)} \}$$

Delta term in output layer with **softmax** activation & **cross-entropic** error is simply: $\delta_i^{(L)} = y_i - \hat{y}_i$ [Per20]

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- Delta rule [Yeh23]

$$\frac{\partial e(\mathbf{y}, \hat{\mathbf{y}})}{\partial \omega_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

$$\delta_i^{(l)} = a_j^{(l-1)} f'^{(l)}(z_i^l) \sum_{j \in N_i^{(l+1)}} \delta_j^{(l+1)} \omega_{ji}^{(l+1)}$$

$$N_i^{(l+1)} = \{ j \text{ indices in layer } (l+1) \mid z_j^{(l+1)} \text{ is connected to } a_i^{(l)} \}$$

Delta term in output layer with **softmax** activation & **cross-entropic** error is simply: $\delta_i^{(L)} = y_i - \hat{y}_i$ [Per20]

- **Backprop** All delta terms $\delta_i^{(l)}$ are computed recursively, starting from **output layer**, going **back** layer-by-layer, until **input layer**

NN technical aspects: algorithms

Back Propagation

Backbone of modern Deep Learning

- Delta rule [Yeh23]

$$\frac{\partial e(\mathbf{y}, \hat{\mathbf{y}})}{\partial \omega_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)}$$

$$\delta_i^{(l)} = a_j^{(l-1)} f'^{(l)}(z_i^l) \sum_{j \in N_i^{(l+1)}} \delta_j^{(l+1)} \omega_{ji}^{(l+1)}$$

$$N_i^{(l+1)} = \{ j \text{ indices in layer } (l+1) \mid z_j^{(l+1)} \text{ is connected to } a_i^{(l)} \}$$

Delta term in output layer with **softmax** activation & **cross-entropic** error is simply: $\delta_i^{(L)} = y_i - \hat{y}_i$ [Per20]

- **Backprop** All delta terms $\delta_i^{(l)}$ are computed recursively, starting from **output layer**, going **back** layer-by-layer, until **input layer**
- **Gradient descent** $\omega_{ij} \leftarrow \omega_{ij} - \alpha \frac{\partial e(\mathbf{y}, \hat{\mathbf{y}})}{\partial \omega_{ij}}$ for all ω_{ij} in the NN

With all the delta terms computed, all the weights ω_{ij} are updated to optimize the error function

NN technical aspects: algorithms

Gradient descent

Strategies

- **Batch Gradient Descent**

- ① compute the mean gradient ∇C across the **entire dataset**
- ② update the model weights

- **Stochastic Gradient Descent (SGD)**

Loop on samples randomly drawn from the dataset:

- ① compute ∇C on each random sample
- ② update the model weights

NN technical aspects: algorithms

Gradient descent

Strategies

- **Batch Gradient Descent**

- ① compute the mean gradient ∇C across the **entire dataset**
- ② update the model weights

- **Stochastic Gradient Descent (SGD)**

Loop on samples randomly drawn from the dataset:

- ① compute ∇C on each random sample
- ② update the model weights

NN technical aspects: algorithms

Gradient descent

Strategies

- **Batch Gradient Descent**

- ① compute the mean gradient ∇C across the **entire dataset**
- ② update the model weights

- **Stochastic Gradient Descent (SGD)**

Loop on samples randomly drawn from the dataset:

- ① compute ∇C on each random sample
- ② update the model weights

- **Mini-Batch SGD**

Build a **mini-batch** (subset of the full dataset)

- ① compute the mean ∇C across the mini-batch
- ② update the model weights

NN technical aspects: algorithms

Gradient descent

Strategies

● Batch Gradient Descent

① compute the mean gradient ∇C across the **entire dataset**

② update the model weights

Method	Data per update	Speed	Stability	Properties
Batch Gradient Descent	Entire dataset	Slow	Very stable	Accurate but computationally expensive
Stochastic Gradient Descent (SGD)	One sample	Very Fast	Noisy	Fast updates with high variance
Mini-Batch SGD	Small subset of data	Fast	Moderately stable	Best trade-off between speed and stability

NN technical aspects: algorithms

Gradient descent

Strategies

- **Batch Gradient Descent**

- ① compute the mean gradient ∇C across the **entire dataset**
- ② update the model weights

- **Stochastic Gradient Descent (SGD)**

Loop on samples randomly drawn from the dataset:

- ① compute ∇C on each random sample
- ② update the model weights

- **Mini-Batch SGD**

Build a **mini-batch** (subset of the full dataset)

- ① compute the mean ∇C across the mini-batch
- ② update the model weights

NN technical aspects: algorithms

tensorflow implementation te training of a Dense Neural Network

- **loss function** : cross-entropy error
- Feed Forward + Back propagation
- **Stochastic Gradient Descent** (Adam) on mini batches

```
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

model = Sequential()
model.add(Input(shape=(100,)))           # INPUT layer
model.add(Dense(64, activation='relu'))   # HIDDEN layer
...
model.add(Dense(10, activation='softmax')) # OUTPUT layer

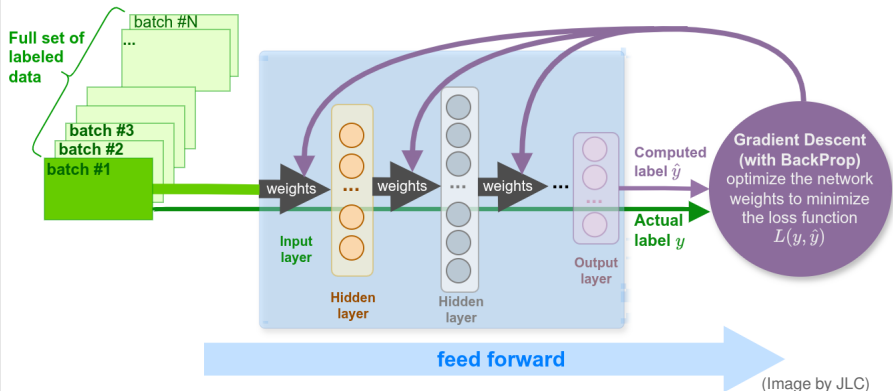
model.compile(loss='categorical_crossentropy', # loss function
              optimizer='adam',               # SGD optimisation
              metrics=['accuracy'])           # output evaluation metrics
model.fit(x_train, y_train,
          epochs=20,                          # repetition of the full training
          batch_size=32                       # mini-batch size of 32 samples
        )
```

ategies

dataset

NN technical aspects: strategy

Supervised learning strategy

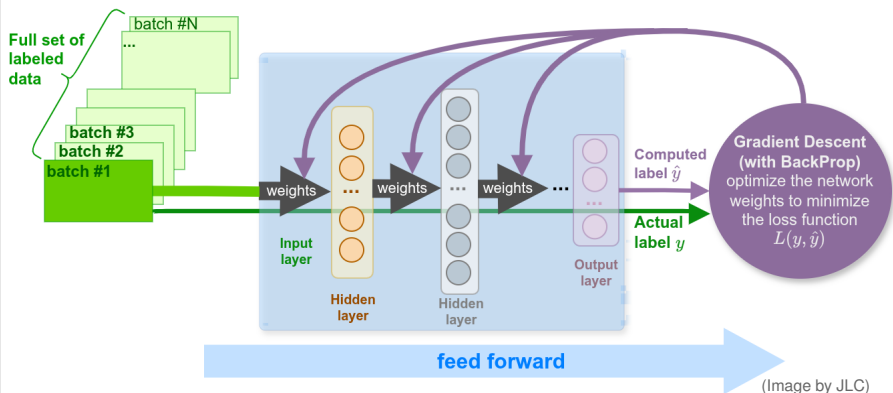


(Image by JLC)

- The dataset is split into (mini) **batches** of size **batch_size**
- *Gradient Descent (with Back Prop to compute ∇L)* modifies the neurons weights to minimize the loss L .

NN technical aspects: strategy

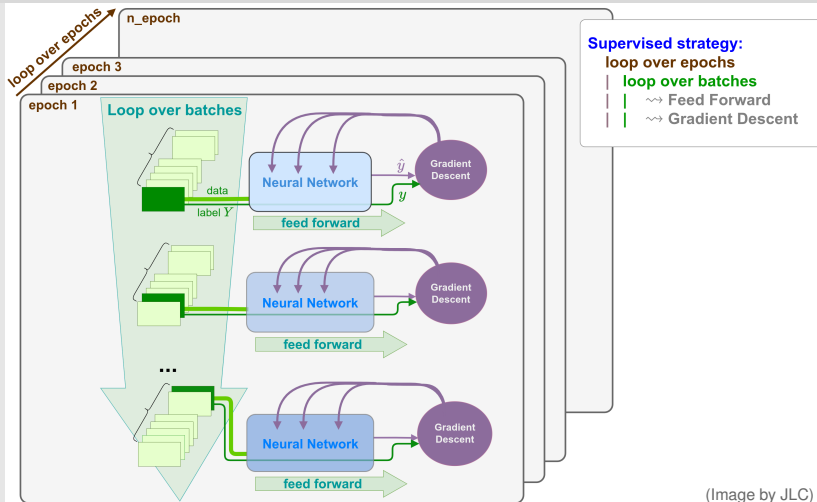
Supervised learning strategy



(Image by JLC)

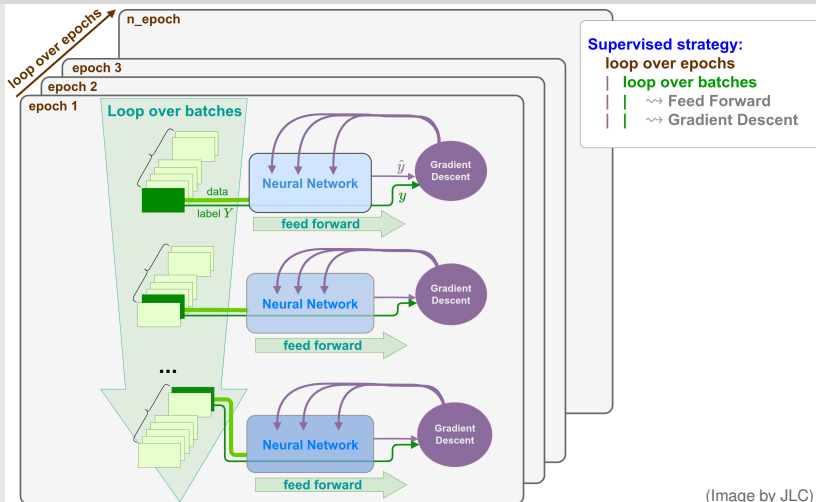
- The dataset is split into (mini) **batches** of size **batch_size**
- *Gradient Descent* (with *Back Prop* to compute ∇L) modifies the neurons weights to minimize the loss L .

NN technical aspects: strategy



- Training with the whole dataset is repeated n_epoch times,
- The Network state after epoch $n \rightsquigarrow$ the initial state for epoch $n+1$.

NN technical aspects: strategy



- Training with the whole dataset is repeated n_epoch times,
- The Network state after epoch $n \rightsquigarrow$ the initial state for epoch $n+1$.

Well known DL Classification example: Training a NN to classify the MNIST handwritten digit images

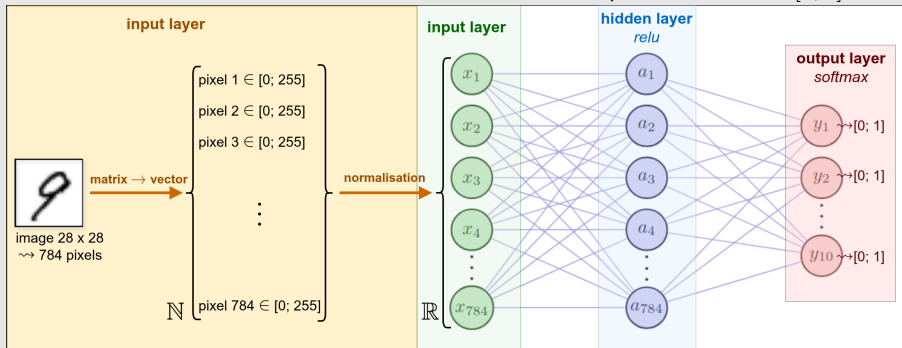
MNIST: 70000 **labeled images**, grayscale, 28×28 pixels
(60000 training images and 10000 test images)



- Classification precision with a Dense Networks can reach 98%
- State of the art for image recognition : **Convolutional Neural Networks** (CNN) or more recently **Vision Transformers** (ViT)
[will not be covered in this course limited to Dense Neural Networks]

Classification with a Dense Neural Network

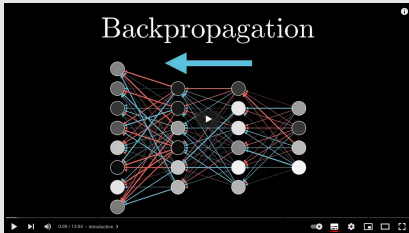
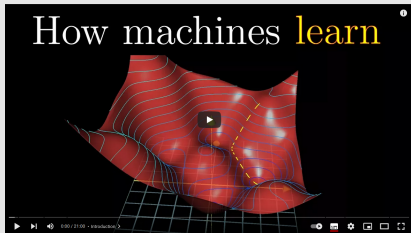
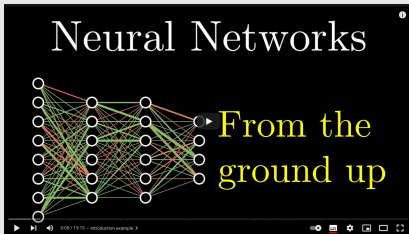
Each matrix $28 \times 28 \rightsquigarrow$ normalized vector of 784 components $\text{float} \in [0; 1]$.



Structure of the neural network:

- The *Input layer* sets the size of the NN inputs to 784 values $\rightsquigarrow$ it has no neuron !
- A *Hidden layer* of 784 neurons (we could have more, or less...), receives the input data. It is connected to the next layer.
- An *Output layer* of 10 neurons (1 neuron for each digit to be recognized).

Videos



Thank you for your attention.

Bibliography I

Artificial Intelligence (AI)

- [Ahm25] S. Ahmed, "Ai agents vs. agentic ai: What's the difference and why it matters," (2025), <https://medium.com/@sahin.samia/ai-agents-vs-agentic-ai-whats-the-difference-and-why-it-matters-8e61d8a43b06> (cit. on p. 22).
- [Ran25] M. K. Ranjan Sapkota Konstantinos I. Roumeliotis, "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges," *Information Fusion*, 2025 (cit. on p. 22).

Deep Learning (DL)

- [AI25] E. AI, "Accuracy, precision, and recall in multi-class classification," (2025), <https://www.evidentlyai.com/classification-metrics/multi-class-metrics>.
- [BR12] J. Bergstra and Y. Rachmad, "Random search for hyper-parameter optimization," *The Journal of Machine Learning Research*, vol. 13, pp. 281–305, Mar. 2012.
- [Cho18] F. Cholet, "Deep learning in python," MANNING, Shelter Island. Downloadable PDF. (2018), <https://archive.org/download/deep-learning-collection-pdf/Deep%20Learning%20with%20Python%20-%20Fran%3CA7ois%20Chollet%20-%20Manning%20%282018%29.pdf>.
- [Cho22] F. Cholet, "Machine learning articles," Articles with tensorflow/pytorch code examples. (2022), <https://github.com/christianversloot/machine-learning-articles/blob/main/README.md>.
- [GBC16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.

Bibliography II

- [He+21] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” (2021), <https://arxiv.org/abs/2111.06377>.
- [MAM24] K. Moulaei, L. Afshari, and R. e. a. Moulaei, “Explainable artificial intelligence for stroke prediction through comparison of deep learning and machine learning models,” *Scientific Reports*, vol. 14, Dec. 2024.
<https://www.nature.com/articles/s41598-024-82931-5>.
- [OM98] G. Orr and K.-R. Müller, “Neural networks : Tricks of the trade,” Published by Springer Berlin Heidelberg, ISBN: 978-3-540-65311-0. (1998), https://archive.org/details/springer_10.1007-3-540-49430-8/mode/2up.
- [Per20] P. Perrotta, “Killer combo: Softmax and cross entropy,” *very good demonstration*. (2020), <https://levelup.gitconnected.com/killer-combo-softmax-and-cross-entropy-5907442f60ba> (cit. on pp. 86–89).
- [SK19] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, volume 6, Article number: 60 (2019). (2019), <https://doi.org/10.1186/s40537-019-0197-0>.
- [sldev25] scikit-learn developers, “Cross-validation: Evaluating estimator performance,” (2025), https://scikit-learn.org/stable/modules/cross_validation.html.
- [SM 30] A. S. S.M. Malakouti M.B. Menhaj, “The usage of 10-fold cross-validation and grid search to enhance ml methods performance in solar farm power generation prediction,” *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, vol. 15, August 20230. <https://doi.org/10.1016/j.clet.2023.100664>.

Bibliography III

- [Sri+14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research* 15 (2014) 1929-1958. (2014), <https://dl.acm.org/doi/abs/10.5555/2627435.2670313>.
- [www26] www.geeksforgeeks.org, "Comparing randomized search and grid search for hyperparameter estimation in scikit learn," (2026), <https://www.geeksforgeeks.org/machine-learning/comparing-randomized-search-and-grid-search-for-hyperparameter-estimation-in-scikit-learn/>.
- [Yeh23] R. Yehoshua, "Backpropagation: Step-by-step derivation," (2023), <https://medium.com/data-science/backpropagation-step-by-step-derivation-99ac8fbddc28> (cit. on pp. 86–89).

Explainable Artificial Intelligence (XAI)

- [Ali+23] S. Ali *et al.*, "Explainable artificial intelligence (xai): What we know and what is left to attain trustworthy artificial intelligence," *Information Fusion*, Volume 99, November 2023, 101805A Preprint. (2023), <https://www.sciencedirect.com/science/article/pii/S1566253523001148> (cit. on pp. 25, 26).
- [MM25] F. Mumuni and A. Mumuni, "Explainable artificial intelligence (xai): From inherent explainability to large language models," *A Preprint*. (2025), <https://arxiv.org/abs/2501.09967> (cit. on pp. 25, 26).

Interpretable Machine learning (IML)

Bibliography IV

- [MCB20] C. Molnar, G. Casalicchio, and B. Bischl, “Interpretable machine learning – a brief history, state-of-the-art and challenges,” Submitted on 19 Oct 2020, published by ECML PKDD 2020 Workshops. ECML PKDD 2020. Communications in Computer and Information Science, vol 1323. Springer, Cham. (2020), <https://arxiv.org/abs/2010.09337> (cit. on pp. 25, 26).
- [Mol24] C. Molnar, “Interpretable machine learning. a guide for making black box models explainable,” book on line. (2024), <https://christophm.github.io/interpretable-ml-book/> (cit. on pp. 25, 26).
- [Rud+21] C. Rudin, C. Chen², Z. Chen¹, H. Huang¹, L. Semenova¹, and C. Zhong¹, “Interpretable machine learning: Fundamental principles and 10 grand challenges,” *Statistics Surveys*, 2021 [last revised 10 Jul 2021 (this version, v2)]. (2021), <https://arxiv.org/abs/2103.11251> (cit. on pp. 25, 26).

Misc programation tips (IML)

- [Cha24] A. Chawla, “A detailed and beginner-friendly introduction to pytorch lightning: The supercharged pytorch,” (2024), <https://www.dailydoseofds.com/a-detailed-and-beginner-friendly-introduction-to-pytorch-lightning-the-supercharged-pytorch>.
- [num25] numpy, “Blas and lapack,” Numpy onmine documentation. (2025), https://numpy.org/doc/stable/building/blas_lapack.html.
- [PyT24] PyTorch, “Use deterministic algorithms,” (2024), https://pytorch.org/docs/stable/generated/torch.use_deterministic_algorithms.html.
- [Pyt24] Pytorch, “Reproducibility,” (2024), <https://pytorch.org/docs/stable/notes/randomness.html#reproducibility>.

Bibliography V

- [Ria19] D. Riach, "Determinism in deep learning (s9911)," GPU Technologie Conference (GTC) 2019 talk. (2019), <https://developer.nvidia.com/gtc/2019/video/s9911>.
- [Ria24] D. Riach, "Framework reproducibility: Determinism (d9m)," (2024), <https://github.com/NVIDIA/framework-reproducibility/blob/master/doc/d9m/README.md>.
- [ten24a] tensorflow, "Enable op determism," (2024), https://www.tensorflow.org/api_docs/python/tf/config/experimental/enable_op_determinism.
- [ten24b] tensorflow, "Set random sed," (2024), https://www.tensorflow.org/api_docs/python/tf/keras/utils/set_random_seed.
- [ten24c] tensorflow, "What's new in tensorflow 2.9?" (2024), <https://blog.tensorflow.org/2022/05/whats-new-in-tensorflow-29.html>.

Misc ressources

- [Gla26] P. Glanze, "Could ai actually wipe out humanity by 2030?" (2026), <https://www.sciencetimes.com/articles/62584/20260911/could-ai-actually-wipe-out-humanity-2030-ai-doomsday-debate-grows-after-anthropic-insider.htm> (cit. on pp. 22–24).
- [LCB10] Y. LeCun, C. Cortes, and C. Burges, "Mnist handwritten digit database," *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, vol. 2, 2010.
- [LHB98] Y. Lecun, P. Haffner, and L. Bottou, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, pp. 2278–2324, Dec. 1998. doi: 10.1109/5.726791.

Bibliography VI

[Tri26]

J. Tripathi, “Best text-to-image ai models 2026: The full comparison,” (August 16, 2026), <https://aizolo.com/blog/best-text-to-image-ai-models-2026-comparison/> (cit. on pp. 36–42).